CareSlot: Implementation Engineer Assessment

Candidate: Boniface Mutisya Ngila

For: Savannah Informatics

Date: 27 October 2026

Summary

CareSlot is a booking API for a clinic with 5 doctors. Patients can see free 30-minute slots, book, cancel and reschedule. It is built with Django REST Framework and PostgreSQL, and runs on Oracle Cloud.

Live API	https://careslot.84-13-134-168.sslip.io/docs
Health check	https://careslot.84-13-134-168.sslip.io/health
Repository	https://github.com/BonifaceNgila/CareSlot

Section	Status
1. System design	Done
2. API	Done, including both bonus features. 43 tests pass.
3. Deployment	Live on OCI over HTTPS
3. CI/CD	PR tests and auto-deploy to OCI are written; not yet run (GitHub billing lock)
4. AI reflection	Done

1. System design

Model	What it holds
Doctor	Name, specialty, active flag
WorkingHours	One shift per doctor per weekday
Patient	Name, optional phone and email
Appointment	Doctor, patient, start and end, status, cancel reason

```
Client -> Caddy (HTTPS) -> Django API -> booking rules -> PostgreSQL
```

Key decisions

- Slots are calculated from working hours, not stored in a table, so they never go out of date.
- All booking rules are in one file, `scheduling/services.py`.
- The database allows only one booked appointment per doctor per start time. If two people book the same slot at once, one gets it and the other gets HTTP 409. A test proves this with 8 bookings at the same moment.
- Cancelled appointments are kept for history, and their slots become free again.

Assumptions

- Time zone is Africa/Nairobi.
- One shift per doctor per day, with no breaks or holidays.
- Bookings need 1 hour's notice.
- Rescheduling keeps the same doctor.
- There is no patient login, so only fictional data is used.

Trade-offs

- A single Django app is simplest for 5 doctors.
- The database rule only blocks identical start times. Variable slot lengths would need a stronger rule.
- Real use would need patient login, backups stored off the server, and more than one server.

2. API

Method and path	Purpose
GET /doctors	List doctors and hours
GET /doctors/{id}/availability?date=	Free slots for a day
POST /patients	Register a patient
POST /appointments	Book a slot
GET /appointments/{id}	View an appointment
PATCH /appointments/{id}/cancel	Cancel with a reason
PATCH /appointments/{id}/reschedule	Move to a new slot
GET /patients/{id}/appointments	Bonus: upcoming appointments
GET /health	App and database check

Errors use one format, with 400 for bad input, 404 for not found and 409 for conflicts:

```
{"error": {"code": "slot_unavailable",
           "message": "This slot is already booked.",
           "details": null}}
```

Try it: the live API has 15 fictional patients and 15 appointments on 28–29 October 2026. Appointment 1 is booked, appointment 2 was rescheduled, and appointment 3 was cancelled. For new bookings, pick a future date and use a slot returned by the availability endpoint.

Tests: 43 tests pass on PostgreSQL 16. They cover booking rules, cancelling, rescheduling, time zones, availability, errors, the 8-way race and the demo data command. Full output: `submission/evidence/postgresql-tests.txt`.

3. Deployment and CI/CD

Hosting: one Oracle Cloud VM (Riyadh) runs three containers: Caddy for HTTPS, the Django API, and PostgreSQL 16. Only ports 80 and 443 are open. Secrets stay on the server. Steps are in `DEPLOY_OCI.md`.

Deploying: `deploy/oci/deploy.sh` builds the image, runs migrations once, restarts the API and checks `/health`. It stops if a migration fails.

Pipeline (`.github/workflows/ci.yml`): every pull request into `main` runs lint, a migration check, all tests on PostgreSQL and a Docker build. A merge into `main` runs the same checks, then:

- copies the code to the server over SSH, using a deploy-only key and a pinned host key
- runs `deploy.sh`
- checks that the live `/health` reports the merged commit

Status: the pipeline is written. GitHub Actions is blocked by a billing issue on the account, so it has not run yet. The live version was deployed by hand with the same script.

Known limits: one server, a temporary IP address, and Ubuntu 20.04, which should be upgraded before real use.

4. AI reflection

What I used AI for: Claude Code and Codex generated most of the code, tests, deployment files and documents. I reviewed the design and booking rules.

An AI suggestion that helped: I asked "Do a dummy data for the API". Instead of raw database rows, the AI wrote a `seed_demo` command that uses the real booking rules, uses only fake data, and doesn't create duplicates when run twice.

AI output that was wrong: the AI's backup check failed with "input file is too short". The backup was fine, but the command didn't pass the file into Docker. Adding `docker exec -i` fixed it. I caught it from the error output.

Decisions I made myself:

- Hosting on OCI, because I know its networking and security setup well.
- Starting with MySQL, because the data is relational and needs transactions. The final version uses PostgreSQL, because the double-booking rule relies on a feature MySQL doesn't have.

Full answers: `REFLECTION.md`.